

Gene Expression Programming-Based Ride Insert Policy for Online Electric Vehicle Ride-Hailing Optimization

Ming-Chu Yang¹, Wei-Neng Chen¹, *Senior Member, IEEE*, Feng-Feng Wei², *Member, IEEE*,
and Jun Zhang³, *Fellow, IEEE*

Abstract—With the popularization of electric vehicles (EVs), the electric vehicle routing problem (EVRP) has gained significant attention. Integrating EVs into ride-hailing services has emerged as a promising trend, but few studies on EVRP comprehensively address key EV ride-hailing features. These features include the constraints of vehicles being unoccupied during charging, the dynamic nature of ride requests, ride-sharing constraints, and so on. Moreover, traditional EVRP algorithms face challenges related to high computational complexity and the need for real-time responsiveness in large-scale and dynamic environments. To address these gaps, this paper introduces the electric vehicle ride-hailing model (EVRHM), a variant of the EVRP designed to incorporate relevant constraints and reformulate the objective function to accommodate EV ride-hailing services better. Subsequently, we propose an online optimization algorithm for the EVRHM based on ride insert policies (RIPs) evolved by genetic programming hyper-heuristic (GPHH). Specifically, a low-level heuristic template is developed to insert dynamic ride requests. When a new ride request is received, the template employs the proposed RIP to insert the ride into the confirmed routes, considering both the current environmental state and ride-specific information. This approach eliminates the need for time-consuming re-optimization processes, enabling real-time responses in dynamic environments. Furthermore, gene expression programming (GEP) within the GPHH framework is designed to learn RIPs, reducing manual effort while improving both the generalization ability and performance of the RIPs. The experimental results demonstrate that the proposed algorithm is effective and efficient in solving EVRHM, with the evolved RIPs exhibiting superior generalization ability.

Index Terms—Electric vehicles and electric mobility, genetic programming, hyper-heuristic, online.

Received 19 February 2025; revised 12 August 2025 and 19 September 2025; accepted 27 September 2025. Date of publication 29 October 2025; date of current version 1 December 2025. This work was supported in part by the National Key Research and Development Project under Grant 2023YFE0206200, in part by the National Natural Science Foundation of China under Grant 62376097, in part by Guangdong Regional Joint Foundation Key Project under Grant 2022B1515120076, and in part by the National Research Foundation of Korea (NRF) grant funded by Korean Government [Ministry of Science and ICT (MSIT)] under Grant RS-2025-00555463. The Associate Editor for this article was H. Yu. (*Corresponding authors: Wei-Neng Chen; Feng-Feng Wei.*)

Ming-Chu Yang, Wei-Neng Chen, and Feng-Feng Wei are with the School of Computer Science and Engineering and the State Key Laboratory of Subtropical Building and Urban Science, South China University of Technology, Guangzhou 510006, China (e-mail: mingcyang@163.com; cschenwn@scut.edu.cn; fengfeng_scut@163.com).

Jun Zhang is with Nankai University, Tianjin 300071, China, also with Hanyang University, ERICA Campus, Ansan 15588, South Korea, and also with Zhejiang Normal University, Jinhua 321017, China (e-mail: junzhang@ieec.org).

Digital Object Identifier 10.1109/TITS.2025.3618704

I. INTRODUCTION

OVER the past decade, greenhouse gas emissions have steadily risen, with transportation contributing over 23% of global emissions [1]. This has prompted efforts to develop sustainable alternatives, such as electric vehicles (EVs), which are becoming increasingly popular in ride-hailing markets [2]. Companies like Cao Cao Chauffeured Car and Jia Bei Sharing Car now operate EV fleets and manage charging networks [3].

Although EVs significantly contribute to mitigating environmental issues, they still face challenges such as lengthy charging times and limited charging facilities [4]. Incorporating EVs into ride-hailing requires managing customer requests and recharging schedules, making the electric vehicle routing problem (EVRP) a popular research topic. Mavrovouniotis et al. [5] formulated the EVRP to minimize total travel distance and introduced a benchmark set to evaluate algorithm performance. Capacitated EVRP (CEVRP) [4] introduces capacity constraints while considering energy constraints to ensure the safety of vehicles. This makes it particularly suitable for carpooling services in ride-hailing operations. Furthermore, systems often need to dynamically plan routes to accommodate new ride requests that arise while pre-planned routes are still being executed. To address this challenge, Basso et al. [6] proposed the Dynamic EVRP (DEVPR). Considering that customers have specific pickup and drop-off points, the EVRP with Pickup and Delivery (EVRPPD) [7] was developed to handle these constraints effectively. Finally, user experience is a critical factor for ride-hailing companies, with customer waiting time being a key performance indicator. To address this, Schneider et al. [8] introduced the EVRP with Time Windows (EVRPTW), requiring vehicles to arrive within customer-specified time windows. Alternatively, Liu et al. [9] proposed a soft time window approach, which allows vehicles to occasionally arrive outside the specified time windows but at the cost of additional penalties, thereby accommodating unforeseen circumstances.

EVRP and its variants are known to be NP-hard [9], [10], [11]. Existing methods for solving EVRPs can be broadly classified into three categories: 1) exact algorithms; 2) individual-based metaheuristic algorithms; and 3) population-based metaheuristic algorithms [12]. Exact algorithms typically model EVRP as a Mixed Integer Linear Programming (MILP) problem [13], [14]. While these methods guarantee optimal solutions, they are computation-

ally infeasible for large-scale instances due to their high complexity. To address large-scale EVRPs, researchers have developed individual-based metaheuristic algorithms [8], [15], [16] and population-based metaheuristic algorithms [12], [17]. These approaches can be highly effective when well-designed heuristics are used, but they struggle to adapt to dynamic environments. In a dynamic environment, the arrival of a new demand triggers an adjustment that must integrate the new demand with the set of previously assigned yet unfulfilled demands, based on the varied vehicle states resulting from prior services. The computational cost of re-optimization, however, makes real-time route adjustment infeasible. To balance response time and computational cost, Montemanni et al. [18] and Kilby et al. [19] proposed dividing the working day into fixed-length time slots, treating the DEVRP as a series of static EVRPs. A time slot represents the interval during which routes are re-optimized. The length of these time slots directly influences the frequency of invoking the optimization algorithm, thereby impacting both system response time and computational resource usage [3]. However, if a new order arrives immediately after the previous optimization is completed, it must wait until the next time slot, leading to delayed responses [20], [21]. This limitation prevents the strategy from effectively handling dynamic orders in real time.

While prior research on the EVRP and its variants has addressed ride-hailing features (e.g., dynamic requests, pickup-delivery pairs, time windows) in isolation, few study have unified these characteristics from a ride-hailing service perspective. Furthermore, existing models, originally developed for logistics applications [5], [9], require adaptation to incorporate ride-hailing-specific constraints, such as passenger-free charging and the strict precedence of pickup points over corresponding drop-off points.

To address these challenges, this paper proposes the EV ride-hailing model (EVRHM) and systematically integrates CEVRP, DEVRP, EVRPPD, and EVRPTW into a unified framework tailored for ride-hailing services, while introducing additional service-specific constraints. Specifically, CEVRP's capacity constraints are adopted to manage ride-pooling demands, while DEVRP is employed to accommodate dynamic ride requests that arrive stochastically, necessitating real-time route adjustments. EVRPPD is utilized to address the requirements of pickup and drop-off points, and EVRPTW introduces soft time windows, thereby balancing customer waiting penalties with operational costs. Meanwhile, to better align with ride-hailing services, two additional constraints are imposed. The first stipulates that each customer's pickup point should be visited before the corresponding drop-off point. The second dictates that vehicles must remain unoccupied while charging at stations. The ride-hailing platform's scheduling system functions as the decision-making entity. It dynamically assigns incoming ride requests to vehicles and plans their itineraries, incorporating service sequences for ride-pooling orders and charging station selections, based on a newly developed objective function. This objective function balances driving costs, customer waiting penalties, and driver working time limits.

The EVRHM poses significant challenges to traditional EVRP solution methods. First, it operates in a dynamic environment that requires high real-time responsiveness. Conventional methods typically rely on re-optimization to handle newly arrived and unfulfilled ride requests, but this process is time-consuming and ill-suited for real-time operation. This difficulty is further exacerbated by the large datasets processed in ride-hailing services. Second, the EVRHM introduces various constraints. Re-optimization can potentially disrupt these constraints, such as the requirement that pickup and drop-off points must be served by the same vehicle. Furthermore, dynamic customer demands may disrupt the previously completed route segments during the re-optimization process [22]. Effectively incorporating such constraints into the re-optimization process remains a non-trivial task.

Considering the limitations of existing methods, this paper proposes an online optimization algorithm that uses ride insert policies (RIPs) evolved by a genetic programming hyper-heuristic (GPHH) for the EVRHM, named GEP-based RIP (GEPRIP). The proposed algorithm establishes an insertion-based low-level heuristic template for constructing routes, which is completely different from the generative construction method used in previous works [23], [24], exhibiting performance limitations. The template leverages the RIP to construct solutions by incrementally inserting each newly arriving request as it appears. During the insertion process, the template avoids modifying any route segments that have been executed prior to the current time. Only insertion positions within the remaining unexecuted segments that satisfy all operational constraints are considered. This approach effectively addresses real-time responses in EVRHM while considering complex operational constraints. Whenever a ride is released, the proposed RIP takes the new ride information and the model's real-time state as inputs, and outputs the appropriate insertion positions for the pickup and drop-off points within the existing routes. Compared to closed box models like reinforcement learning, whose internal decision-making processes remain opaque, the RIPs generated by GEPRIP possess inherent interpretability [25]. However, designing an effective RIP is a challenging task, as it requires specialized knowledge to identify the key attributes that reflect the problem's features. Moreover, since the topology of each instance can vary, distinct RIPs may need to be designed for different instances. Gene expression programming (GEP) [26], a refined variant of GP, is employed within GPHH to explore the low-level heuristic space of RIPs, owing to its global search capability and the generalizability of the evolved rules [27]. It demonstrates significant potential in dynamic and uncertain environments [27], [28], [29]. The main contributions of this paper are described as follows:

- A new variant of the EVRP termed the **Electric Vehicle Ride-Hailing Model (EVRHM)** is introduced to address the unique requirements of ride-hailing services. This model effectively integrates a range of constraints, including vehicle capacity, battery levels, pickup and drop-off requirements, time windows, dynamic ride requests, and additional ride-hailing-specific constraints. Furthermore, a new objective function is defined to optimize key

ride-hailing metrics, such as driving costs, driver working hours, and customer waiting penalties.

- An online optimization algorithm for the EVRHM is proposed. It establishes a low-level heuristic template to transform the iterative optimization process used in traditional algorithms into an efficient insertion process. When a new ride is requested, the scheduling template utilizes RIP to determine the insertion position of pickup and drop-off points for the ride within existing routes, by considering the ride information and the model's current state. In this way, we only need to consider the latest ride information and model state without optimizing all routes, which avoids time-consuming re-optimization processes, thereby effectively handling dynamic ride requests.
- A GPHH is designed to evolve effective low-level heuristics as RIPs. Specifically, GEP is employed in place of traditional GP to enhance the evolutionary process. Leveraging its unique encoding and expression capabilities, GEP improves the generalization and performance of the evolved RIPs [26]. Additionally, a primitive set is designed for the RIPs from the operator's global perspective, tailored to the characteristics of EVRHM. Consequently, the scheduling decisions guided by RIPs exhibit superior globality and foresight compared to general heuristics [29]. Through searching the low-level heuristic space defined by the primitive sets, GEP automatically generates expressions of arbitrary complexity. This approach not only reduces the manual effort required but also enhances the efficiency and accuracy of the generated expressions.

To validate the effectiveness of the proposed algorithm, a series of experiments were conducted to enable comprehensive comparisons and analyses. The experimental results demonstrate that GEPRIP can solve the EVRHM efficiently and effectively while exhibiting strong generalization capabilities.

The remainder of this paper is organized as follows. Section II reviews the related work on EVRP and GPHH. Section III presents the formulation of the EVRHM. The proposed GEPRIP algorithm is introduced in Section IV. Section V details the experimental setup and results. Finally, Section VI concludes the paper.

II. RELATED WORK

A. EVRP and Its Variants

Researchers have focused on the EVRP to support the green transition in logistics and distribution by promoting the replacement of fuel-powered vehicles with EVs [30].

In 2011, Conrad and Figliozzi [31] were the first to integrate vehicle route planning with charging schedules, proposing charging at customer locations to alleviate range anxiety. Their study analyzed key factors such as battery capacity and charging time, providing valuable insights for EV route planning. However, the assumption that charging occurs at customer locations deviates from real-world logistics practices. Building on this, Erdoğan and Miller-Hooks [32] introduced the green vehicle routing problem, which is considered the precursor of EVRP as the first model to incorporate alternative

fuel vehicles in logistics. They developed two heuristic algorithms and established standard problem instances, showing that driving range limitations significantly increase the number of refueling stops and total travel distance. Schneider et al. [8] later proposed EVRPTW, which incorporates time windows for customer service. However, its rigid constraints are often impractical for real-world scenarios. To address this, Liu et al. [9] developed a model for EVRPTW with soft time windows, minimizing total costs including vehicle, energy, and penalty costs for delayed deliveries. Since charging EVs is time-intensive, customer service times in EVRPTW require greater consideration compared to VRPTW. For pickup and delivery problems, vehicles must pick up and deliver transport loads from a given origin to a destination [33]. Chen et al. [7] extended this problem by modeling EV fleet operations with small battery capacities from a transit service provider's perspective. Yilmaz and Kalayci [34] formulated EVRPPD as an integer linear model, incorporating simultaneous delivery and pickup tasks. Most EVRP studies assume that customer requests are known in advance [7], [8], [9]. However, in practice, customer requests often emerge stochastically and require prompt responses. Wang et al. [35] studied DEVRP, proposing an adaptive large neighborhood search (ALNS) method capable of handling up to 1000 requests. Despite its merits, their approach relies on periodic re-optimization, which may not meet the demands for immediate responses to dynamic customer requests. Although extensive research has been conducted on EVRP and its variants, these models are not well-suited for ride-hailing services, which require more dynamic and customer-oriented solutions. Table S1 in the supplementary material shows the relations between the relevant models and the EVRHM.

B. Algorithms for EVRPs

The methods proposed for EVRPs can be broadly classified into three categories: 1) exact methods, 2) individual-based metaheuristic methods, and 3) population-based metaheuristic methods [12]. Table S2 in the supplementary material summarizes the characteristics and respective limitations of the three types of algorithms. Exact methods, such as those using mathematical programming software like CPLEX and Gurobi [7], [14], are capable of generating optimal solutions for the EVRP. For example, Chen et al. [7] modeled the EVRPPD as a mixed-integer quadratically constrained programming problem, solving instances with up to 20 customers. However, while these exact methods are effective for small-scale problems, they face significant efficiency challenges when applied to large-scale problems. Individual-based metaheuristic methods, including variable neighborhood search [8], iterative local search [36], and adaptive large neighborhood search [15], [34], have been widely used to solve large-scale EVRPs. However, these methods generate only a single solution during computation, making them highly sensitive to the quality of the initial solution. If the perturbation operators are poorly selected or the initial solution is suboptimal, the algorithm may easily converge to poor local optima [37]. Due to their strong global search capabilities, population-based metaheuristic algorithms have also been widely applied. For instance,

Zhenfeng et al. [38] employed genetic algorithms (GA) for EVRPTW. However, the method assumes that each charging station can be visited only once, limiting its effectiveness to small-scale problems. Similarly, ant colony optimization (ACO) has been applied to address the EVRP [4], [9], [12], but the simultaneous consideration of customer service orders and vehicle recharging schedules results in a complex and large search space [39]. Given that current algorithms for dynamic ride characteristics in EVRHM require significant computational resources, achieving real-time responsiveness remains a challenging task.

C. Genetic Programming Hyper-Heuristic

Hyper-heuristics is a heuristic search method that aims to efficiently solve computational search problems by automatically selecting or generating heuristics, such as those encountered in timetabling and scheduling problems [40]. A hyper-heuristic algorithm searches for the best heuristic combinations within the space of heuristic algorithms to produce solutions of acceptable quality. If a hyper-heuristic algorithm employs GP to generate low-level heuristics, it is referred to as GPHH. Koza [41] introduced GP as an extension of genetic algorithms (GA). GP represents a computer program or a function as a parse tree. By specifying the appropriate function set and terminal set, it can automatically evolve the tree to meet specific requirements [27]. As a generative hyper-heuristic algorithm, GP has gained considerable popularity due to its capability for global automatic optimization within the search space [27], [29], [42]. However, existing studies on the application of GP to EVRP remain limited, with only two known examples [23], [24]. Both studies focus exclusively on static scenarios and demonstrate a notable performance gap in solution quality compared to state-of-the-art algorithms. Consequently, GP has received relatively limited attention in EVRP. Nonetheless, it holds significant potential in addressing DEVRP, where its flexibility and adaptability could be advantageous.

GEP is a well-known variant of GP. The chromosomes in GEP are simple entities: linear, compact, relatively small, and easy to manipulate genetically [26]. Therefore, GEP has enhanced computational solving capabilities and extensive applicability. For instance, Zhang and Liu [43] combined the tabu search algorithm with GEP. This combination was successfully applied to the prismatic leaching test of rare earths. Liao et al. [29] also adopted GEP to train a reactive assignment strategy, which was applied to solving the dynamic version of the traffic assignment problem.

III. THE FORMULATED EVRHM

This section presents an introduction to EVRHM, providing the necessary background and context. It then formulates the corresponding problem model, highlighting the key constraints and objectives.

A. Problem Introduction

EVRHM is defined on a weighted fully connected graph $G = (V, E)$, where $V = \{0\} \cup V_c \cup V_f$ is the set of nodes and E

is the set of arcs. The set $\{0\}$ represents the depot, which serves as the starting and ending point for all EVs. The customer nodes are denoted by V_c , while V_f represents the charging stations. Each ride (customer) consists of a pickup point and a drop-off point, such that $V_c = V_p \cup V_q = \{1, 2, \dots, 2n\}$, where n is the number of ride requests. The set $V_p = \{1, 3, \dots, 2n-1\}$ denotes pickup points, and $V_q = \{2, 4, \dots, 2n\}$ denotes drop-off points. For ride i , the pickup point is labeled $2i-1$, and the drop-off point is labeled $2i$. To accommodate recharging requirements, $\hat{V}_f$ is defined as the set formed by replicating each charging station $|V_c|$ times, unlike EVRP, where each station is replicated $2|V_c|$ times [44]. Since vehicles cannot carry passengers while at charging stations, the worst-case scenario assumes that each customer requires a dedicated EV. In this scenario, each vehicle may need to charge once before reaching the pickup point and once after completing the drop-off.

Each node $i \in V_c$ is associated with a fixed demand, denoted as c_i . For each pickup point $p_i \in V_p$, the demand c_{p_i} is defined as a positive number, representing a customer who needs to board. In contrast, each drop-off point $q_i \in V_q$ has a negative demand, denoted as c_{q_i} , reflecting the passenger being dropped off. The demand of a pickup point is the negative of its corresponding drop-off point, i.e. $c_{2i-1} = -c_{2i}$.

Each arc $(i, j) \in E$ is associated with a non-negative distance value d_{ij} , representing the distance between nodes i and j . An EV is subject to both a load limit Q_c and a battery capacity limit Q_b . The battery consumption rate is defined as r_b , indicating that an EV will consume an amount of battery equal to $r_b \cdot d_{ij}$ when traversing arc (i, j) .

The EVRHM can be described as follows: It seeks to determine the optimal routes for a homogeneous fleet of EVs, under the constraints of both battery and load limitations, to fulfill the ride-hailing demands of a set of customers, with all vehicles starting from and returning to the central depot.

B. Problem Formulation

A new objective function is formulated to adapt to EVRHM, considering three key factors: 1) Driving cost: This includes both the total travel distance and the number of EVs dispatched from the depot, which contribute to operational overhead. 2) Driver overtime penalty: To prevent driver fatigue, the maximum working time for each driver is set to 480 minutes. If the maximum working time is exceeded, no new rides are assigned to the driver. However, if the driver has not completed all the rides within the specified time, they are still required to finish them. 3) Ride overdue penalty: This is an important service quality indicator. The latest completion time for ride i is calculated as: $l_i = (d_{0p_i} + d_{p_iq_i})/v + 20$, where 0 denotes the depot, p_i and q_i represent the pickup and drop-off points of ride i , respectively, and v represents the velocity of the homogeneous vehicle. To address the unpredictability of ride requests in a dynamic environment, slight overtime is allowed for both the driver's working time and the customer's waiting time. However, this will result in certain penalties. Hence, the overall model can be formulated as follows:

$$\min F = (1 + (\rho_1 P_r + \rho_2 P_d + N_r) / N_{req}) \cdot f \quad (1)$$

$$f = \sum_{k \in K} \sum_{j \in V, i \neq j} \sum_{i \in V} d_{ij} x_{ij}^k \quad (2)$$

$$\text{s.t. } \sum_{k \in K} \sum_{j \in V} x_{ij}^k = 1 \quad \forall i \in V_c \quad (3)$$

$$\sum_{k \in K} \sum_{j \in V} x_{ij}^k \leq 1 \quad \forall i \in \hat{V}_f \quad (4)$$

$$\sum_{j \in V} x_{ij}^k - \sum_{j \in V} x_{ji}^k = 0 \quad \forall i \in V \quad \forall k \in K \quad (5)$$

$$u_j \leq u_i - c_i x_{ij}^k + Q_c (1 - x_{ij}^k) \quad \forall i \in V \quad \forall j \in V \quad \forall k \in K \quad (6)$$

$$0 \leq u_i \leq Q_c \quad \forall i \in V \quad (7)$$

$$y_j \leq y_i - r_b d_{ij} x_{ij}^k + Q_b (1 - x_{ij}^k) \quad \forall i \in V_c \quad \forall j \in V \quad \forall k \in K \quad (8)$$

$$y_j \leq Q_b - r_b d_{ij} x_{ij}^k \quad \forall i \in \hat{V}_f \cup \{0\} \quad \forall j \in V \quad \forall k \in K \quad (9)$$

$$0 \leq y_i \leq Q_b \quad \forall i \in V \quad (10)$$

$$x_{ij}^k \in \{0, 1\} \quad \forall i \in V \quad \forall j \in V \quad \forall k \in K \quad (11)$$

$$\sum_{j \in V} x_{ij}^k - \sum_{j \in V} x_{i+1j}^k = 0 \quad \forall i \in V_p \quad \forall k \in K \quad (12)$$

$$u_i = Q_c \quad \forall i \in \hat{V}_f \quad (13)$$

The objective function of the model is defined by (1), where P_r , P_d , and N_r represent the number of overdue rides, the number of overtime drivers, and the number of dispatched EVs, respectively. The variable N_{req} denotes the number of ride requests (customers), set as $|V_p|$. Additionally, ρ_1 and ρ_2 are pre-defined penalty coefficients for customer overdue and driver overtime, respectively, designed to balance the overtime situations of customers and drivers. In this study, $\rho_1 = 1.5$ and $\rho_2 = 1.2$. The function f calculates the total travel distance, as illustrated in (2), where x_{ij}^k is the decision variable. If EV k travels arc (i, j) , then $x_{ij}^k = 1$; otherwise, $x_{ij}^k = 0$. Finally, K represents the fleet of homogeneous EVs. A weighted sum is applied to the three additional penalty costs (P_r , P_d , N_r). The resulting penalty cost is then normalized through division by N_{req} to ensure scalability and independence from the problem size. Finally, the objective function multiplies the base cost f by a coefficient of $(1 + \text{normalized penalty cost})$, a multiplicative structure that prioritizes minimizing travel distance while penalizing other operational inefficiencies.

EVRHM must satisfy the basic constraints defined in [5], which are formulated in (3)-(11). Constraint (3) ensures that each customer is served exactly once. Constraint (4) limits each copy of a charging station to be visited at most once. Since each charging station has $|V_c|$ copies, (4) effectively indicates that any EV can visit charging stations (including the depot) multiple times. Constraint (5) ensures flow conservation by guaranteeing that the in-degree (vehicle entering) of a node is equal to its outdegree (vehicle leaving). Two additional variables, u_i and y_i , are introduced to represent the remaining load capacity and remaining battery charge level of an EV upon its arrival at node i , respectively. Constraints (6) and (7) impose vehicle capacity constraints, ensuring that each customer's demand is fully met. Constraints (8)-(10) define vehicle battery constraints, with (9) specifying that EVs always leave charging stations fully charged (the depot is also

considered as a charging station). Finally, constraint (11) defines a set of binary decision variables.

It is important to note that in addition to meeting the core EVRP constraints outlined above, EVRHM must also account for specific constraints unique to the ride-hailing service. These additional constraints are defined in (12) and (13). Constraint (12) ensures that the pickup and drop-off locations of the same customer are served by the same vehicle, while (13) prevents customers from being in the EV during charging. Furthermore, another key constraint is that the pickup point must be visited before the drop-off point for each ride. Although these constraints are not explicitly addressed in the EVRP, they are essential for the formulation of EVRHM. An example of EVRHM is provided in Fig. 1.

The model discussed above refers to a deterministic version of EVRHM, where all ride requests are known in advance. However, in practice, ride requests appear dynamically within EVRHM. EVRHM can be conceptualized by decomposing it into a series of deterministic instances. Each instance is triggered by a newly released ride request and considers all unvisited known requests along with the new request. It is crucial to note that once a pickup point is serviced by an EV, the corresponding drop-off point must also be serviced by the same EV during the re-optimization process. Furthermore, a key assumption is that an EV, while en route to a node, cannot alter its direction to another node. This assumption stems from the understanding that computing a new route for a vehicle in motion is time-consuming, and frequent changes in vehicle direction can lead to a poor user experience.

IV. THE PROPOSED GEPRIP

This section provides a detailed introduction to the proposed GEPRIP. First, a low-level heuristic template is presented, which generates feasible solutions for a given RIP and an EVRHM instance with a ride queue using an insertion-based approach. Next, the evolution process of the RIP is described, highlighting how the GEPRIP framework explores the search space of RIPs, rather than directly seeking solutions. Finally, a primitive set designed for evolving the RIP is introduced.

A. Low-Level Heuristic Template

The low-level heuristic RIP serves effectively as a priority function, incrementally inserting components to construct a feasible solution. The heuristic template simulates the component insertion process, taking into account the relevant constraints.

Given an EVRHM instance with a queue of ride requests, denoted as $\mathcal{Q}$, and a RIP $\mathcal{R}$, the objective is to generate an efficient solution, $\mathbb{S}$, that satisfies customer demands. $\mathbb{S}$ can be defined as a set of vehicle routes based on x_{ij}^k . Each route consists of a sequence of nodes, representing the order in which the vehicle visits the locations. For example, the solution shown in Fig. 1 can be represented as follows:

$$\mathbb{S} = [K_1, K_2] \quad (14)$$

where

$$K_1 = (0, 1, 2, 9, 5, 6, 0) \quad (15)$$

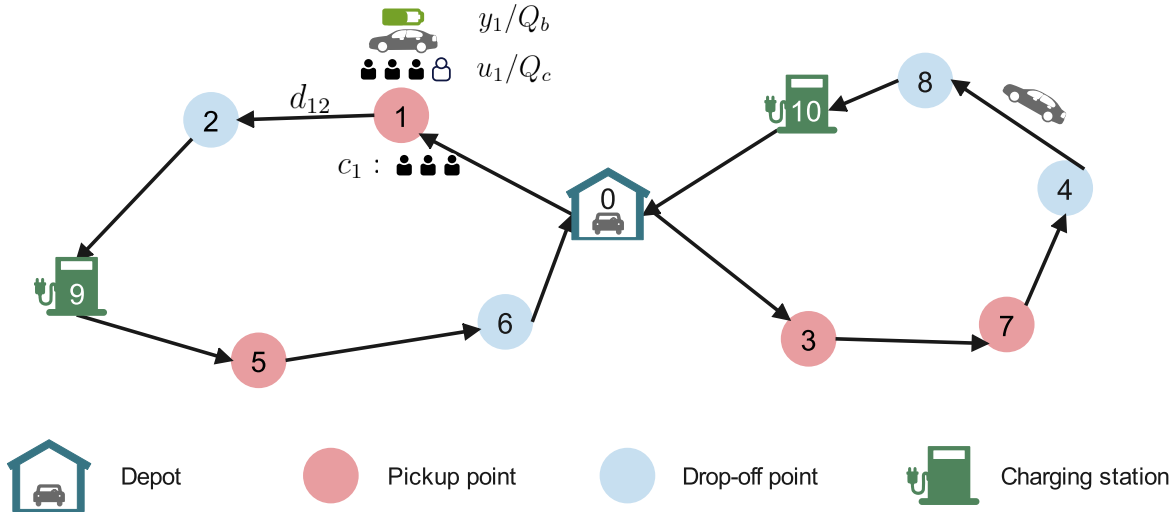


Fig. 1. An example solution of EVRHM with two routes.

$$K_2 = (0, 3, 7, 4, 8, 10, 0) \quad (16)$$

Additionally, information for each route, such as arrival times, battery levels, and remaining capacity at each location, must also be recorded in $\mathbb{S}$.

Rides in $\mathcal{Q}$ are sorted based on the ride publication time, t_r , with earlier rides placed at the front. Due to the dynamic nature of EVRHM, ride details—including the pickup point p_r , drop-off point q_r , and customer demand c_r —are only known when the ride is requested. Upon receiving a ride request, the heuristic template determines the appropriate insertion positions for both the pickup and drop-off points. The pseudocode for the proposed template is shown in Algorithm 1.

Step 1 (Initialization): $\mathbb{S}$ is initialized as a set containing only an empty route (line 1). In this paper, the empty route is denoted as $\emptyset = (0, 0)$. For the convenience of pseudocode representation, the presence of the empty route $\emptyset$ in $\mathbb{S}$ is necessary to facilitate the simulation of a direct departure from the depot.

Step 2 (Ride Information Update): As long as $\mathcal{Q}$ is not empty, the latest ride r is dequeued (line 3), representing the process of ride publication. Next, we update the relevant information of ride r and initialize the best priority value, o^* , as $+\infty$ (lines 4 and 5).

Step 3 (Insertion Locations Traversal): The main traversal process (lines 6-15) consists of a triple nested loop that identifies the appropriate insertion locations for the pickup and drop-off points of the newly published ride. This is done by traversing all possible insertion positions according to $\mathcal{R}$.

The outer loop iterates over each route K in $\mathbb{S}$ (line 6). First, determine the current location index l_k of the vehicle in route K at time t_r (line 7). If the vehicle is en route to a node at time t_r , then l_k corresponds to the index of that node in K . Otherwise, l_k represents the index of the node in K where the vehicle is currently located.

The middle loop iterates over the possible insertion locations i_p for p_r , ranging from l_k to the length of K (line 8). If the constraints are satisfied after inserting p_r at i_p , the index of the next charging station j_i , located after i_p in K , is updated

(lines 9 and 10). In this context, the depot is also treated as a charging station.

The inner loop iterates over the insertion positions i_q for q_r , ranging from $i_p + 1$ to j_i , which ensures that no passengers are in the vehicle during charging. Since i_q must follow i_p and the customer demand at the drop-off point q_r is negative, we only need to consider the energy constraint (lines 11 and 12).

Step 4 (Priority Value Calculation): If the energy constraint is still satisfied by adjusting charging stations after inserting p_r and q_r , the status features z are updated following the insertion, and the priority value o is calculated according to $\mathcal{R}$ (lines 13 and 14). If o is smaller than o^* , we update o^* and the corresponding insertion positions (line 15).

Step 5 (Solution Update): After determining the insertion positions, the algorithm updates $\mathbb{S}$ and inserts the charging stations (lines 16 and 17). The strategy employed in this study is to insert charging stations only when all rides assigned to the vehicle have been completed and the power level is below a certain threshold. Alternatively, if all previous rides have been completed but the remaining power is insufficient to complete the newly assigned ride, the vehicle will visit the charging station before reaching the pickup point of the new ride. In this study, it is assumed that the vehicle is fully charged each time it visits a charging station, and that the charging time is proportional to the amount of charge. Given that EVRHM operates in a dynamic environment, frequently optimizing the position of charging stations may not be practical for ensuring efficient pickup and drop-off, and could also consume significant computational resources. Therefore, charging when the power is low is more realistic in the context of this study. Accordingly, we update the relevant information for route K^* (line 18) in preparation for the insertion of the next ride.

Repeat the above steps until the ride queue is empty, and then return the solution (line 19).

B. RIP Evolution Process

Designing RIPs manually not only requires specialized knowledge but also involves creating various RIPs tailored to

Algorithm 1 Low-Level Heuristic Template

input : an EVRHM instance with a ride queue Q ,
a RIP $\mathcal{R}$
output: an effective solution $\mathbb{S}$
// **Initialization**
1 $\mathbb{S} \leftarrow [\emptyset]$;
2 **while** Q is not empty **do**
 // **Ride Information Update**
 3 Dequeue r from Q ;
 4 Update r information t_r, c_r, p_r, q_r ;
 5 $o^* \leftarrow +\infty$;
 // **Insertion Locations Traversal**
 6 **for each** route K in $\mathbb{S}$ **do**
 7 Determine the current location index l_k in K at
 t_r ;
 8 **for** insert index $i_p \leftarrow l_k$ to $|K|$ **do**
 9 **if** the constraints are satisfied after
 inserting p_r **then**
 10 Update charging station index j_i ;
 11 **for** insert index $i_q \leftarrow i_p + 1$ to j_i **do**
 12 **if** the energy constraint is satisfied
 by adjusting charging stations
 after inserting p_r and q_r **then**
 // **Priority Value Calculation**
 13 Get the status features z after
 inserting p_r and q_r ;
 14 Calculate the priority value
 $o \leftarrow \mathcal{R}(z)$;
 15 **if** $o < o^*$ **then** $K^* \leftarrow K$,
 $o^* \leftarrow o, p^* \leftarrow i_p, q^* \leftarrow i_q$;
 // **Solution Update**
 16 Insert p_r and q_r based on K^*, p^* and q^* ;
 17 Insert charging station;
 18 Update information of route K^* ;
19 **return** $\mathbb{S}$;

different problem topologies. Automatically generating RIPs based on GEP within GPHH can significantly reduce manual effort and potentially produce novel RIPs that humans may not consider [29].

In GEP, chromosomes consist of one or more genes of equal length. Each gene can be converted into an equivalent expression tree that represents a mathematical formula. A gene is composed of a fixed-length string, which is divided into a head and a tail. The symbols in the head are selected from the function set and terminal set, while the tail contains only terminals. The function set and terminal set are essential components of the gene, each with predefined elements. The function set includes operators relevant to the problem, while the terminal set contains constants and problem-specific inputs.

As shown in Fig.2, the single-gene chromosome can be transformed into an equivalent expression tree using a breadth-first traversal schema. Its corresponding expression is:

$$a - \frac{d}{2} + bc \quad (17)$$

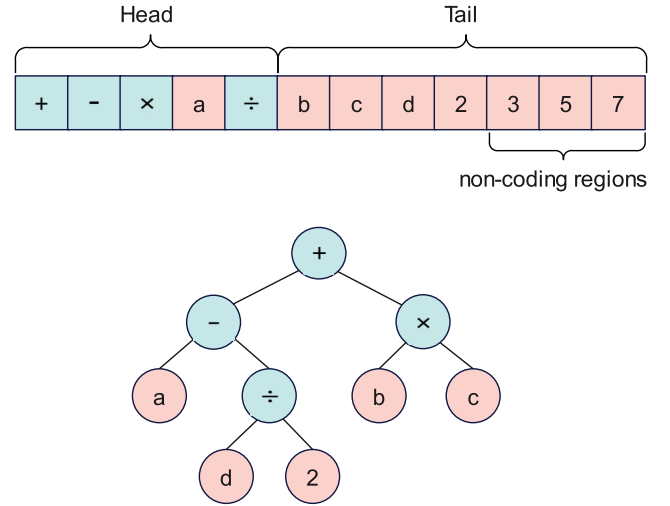


Fig. 2. Typical single-gene chromosome and its decoded expression tree. The green nodes denote functions and the red nodes represent terminals.

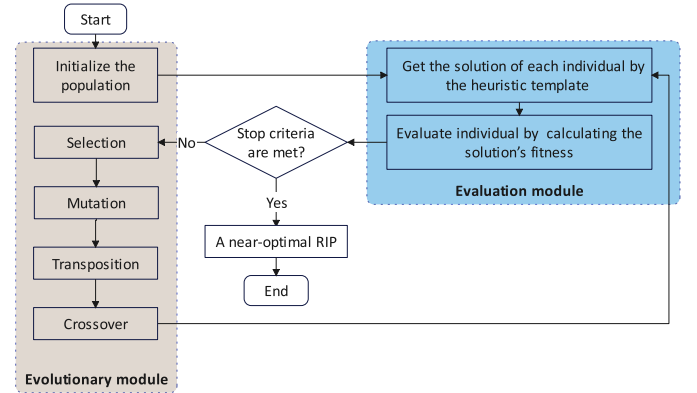


Fig. 3. Framework of GEP.

Although non-coding regions do not directly contribute to expression formation, the flexible gene expression method ensures that solutions generated during the algorithm's evolution always adhere to the rules. The unique structure of GEP genes guarantees that each gene can be mapped to an expression tree, resulting in a complete and valid expression.

As shown in Fig.3, the GEP framework begins with an initial population consisting of randomly generated individuals. Each individual represents a candidate RIP and must be evaluated. The heuristic template described above generates the corresponding solution based on the RIP. The fitness of each solution is calculated using (1) to evaluate the individuals. After evaluating all individuals, a series of genetic operators are applied to evolve the population, including 1) selection, 2) inversion mutation, 3) point mutation, 4) insertion sequence (IS) transposition, 5) root IS transposition, 6) gene transposition, 7) one-point crossover, 8) two-point crossover, and 9) gene crossover. These operations follow the standard methods described in [26]. The process continues until the stopping criteria are met, and the best individual from the final population is returned as the effective RIP.

C. RIP Primitive Set

To adapt GEP to a specific problem, the most crucial step is defining the terminal set and function set, which are used to construct the RIP.

The terminals in the problem should reflect the model's state and key aspects of the problem domain to generate high-quality RIPs. However, an excessive number of terminals should be avoided, as it would result in an overly large heuristic search space and longer training times. Moreover, overly complex terminals may lead to overfitting, which can negatively affect the generalization performance of the RIP.

In this study, the terminal set TS is designed as:

$$TS = z = \{f, P_r, P_d, NT, N_r\} \quad (18)$$

where f , P_r , P_d , and N_r directly influence the objective function (1), and NT denotes the time required to complete the rides already assigned to the insertion vehicle. These terminals represent the basic state features z of the solution after inserting the pickup and drop-off points at the corresponding locations. More complex terminals, defined as combinations of simpler ones, are avoided to prevent premature convergence to local optima.

The function set FS is defined as:

$$FS = \{+, -, \times, \div, \min, \max, \exp, \log, \sqrt{\cdot}\} \quad (19)$$

where $+$, $-$, $\times$, $\div$, $\min$ and $\max$ operate on two arguments, with function $\div$ ensuring protected division and returning one in case of division by zero. The functions $\exp$, $\log$, and $\sqrt{\cdot}$ each take a single argument, with $\log$ using e as the base. If the argument a is negative, $\log$ and $\sqrt{\cdot}$ return $-\log(-a)$ and $-\sqrt{-a}$, respectively.

D. Time Complexity

The GEPRIP algorithm mainly comprises two stages: the GEP-based RIP evolution stage and the RIP application stage. In practical applications, the RIP is trained only once for each network, making the training process one-off. During actual deployment, only the low-level heuristic template needs to be executed, and RIP retraining is not necessary. The time complexities of both stages are analyzed in detail below.

The time complexity of the GEP-based RIP evolution is determined by the following key factors: generation number G , population size P , the complexity of fitness evaluation F (defined as the product of the training data size and the time required to execute a single low-level heuristic template, as will be discussed later), and the cost of evolutionary operations D [45]. The overall time complexity of training can be expressed as $O(G \times P \times (F + D))$. Since F is significantly higher than D , the overall complexity is mainly dominated by F . Therefore, it can be approximated as $O(GPF)$.

Our algorithm is an online algorithm, characterized by its ability to respond in real-time to dynamically arriving ride requests. Accordingly, we primarily focus on the time complexity of executing the low-level heuristic template for inserting a newly arrived ride request. The low-level heuristic

template is adopted to insert ride points for incremental solution construction. When inserting each ride request (involving 2 nodes), two steps are involved: traversing all possible insertion positions across all vehicles, and checking whether the path constraints are satisfied after insertion. Iterating through all possible insertion positions involves a triple-nested loop, leading to a worst-case time complexity of $O(KL^2)$, where K denotes the number of vehicle routes and L is the average number of nodes per route. When traversing each insertion position, updating the system state and checking the validity of route constraints incur a worst-case time complexity of $O(L)$. Therefore, the worst-case time complexity for inserting a single request is $O(KL^3)$. In practical applications, we only need to consider currently operating vehicles in the vicinity of the request, whose number, denoted as k ($k \ll K$), can be regarded as a constant. For each vehicle, insertion positions only need to consider ride nodes already assigned but not yet completed. Due to capacity constraints (set to 4 in this study) and the risk of timeouts when too many passengers are assigned to a single vehicle, the number of nodes l that each vehicle can serve at any given time (i.e., the number of insertion positions to consider) is limited, $l \ll L$, and can be regarded as a constant. Consequently, the actual complexity for inserting a single request is $O(kl^3)$, where both k and l are typically small and can be treated as constants in practical scenarios.

V. EXPERIMENTS

In this section, the proposed GEPRIP is evaluated on eight instances from two real-world road networks. First, the process of generating the benchmark set for EVRHM is described in detail. Next, the experimental settings are outlined, and the performance of GEPRIP is compared with other algorithms to demonstrate its high performance and good generalizability. Finally, an analysis of the different genetic operations within GEP is presented.

A. Benchmark Set

Since EVRHM is a new model, no benchmark sets are currently available for comparison and validation. To advance research on EVRHM, a benchmark set based on real-world data is presented.

Two real-world road networks, from Beijing and Chengdu, are designed for the experiments. The node heat maps of the two networks are shown in Fig. 4, and the data are obtained through the map generator and coordinate pickup tool available at <https://lbs.amap.com/>. The Beijing network consists of 615 nodes, including 1 depot, 14 charging stations, and 600 pickup and drop-off points (300 each), with customer nodes exhibiting a clustered distribution. The Chengdu network has a total of 413 nodes, comprising 1 depot, 12 charging stations, and 200 rides (corresponding to 400 nodes). The distribution of customer nodes is more concentrated in the city center, with a smaller coverage area compared to Beijing.

In practice, most rides typically accommodate no more than four passengers. Thus, the customer demand c_i for each pickup point $i \in V_p$ is randomly generated from the set

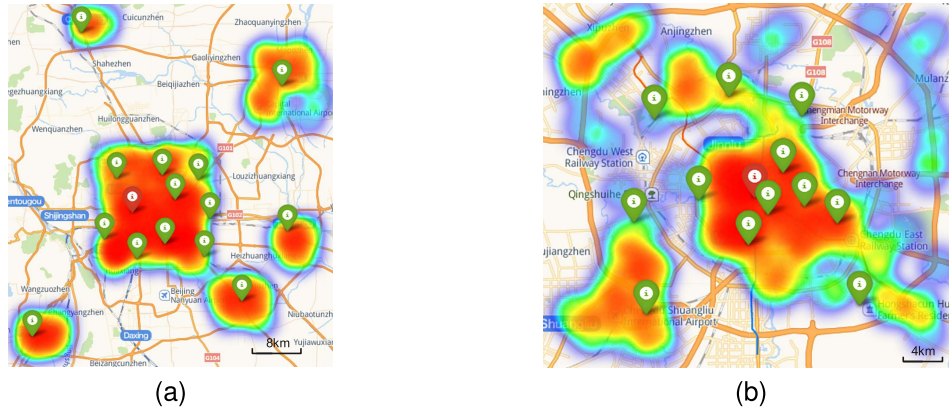


Fig. 4. Node heat map. The red icons indicate depots and the green icons denote charging stations. (a) Beijing. (b) Chengdu.

{1, 2, 3, 4}. The load capacity of EVs is set to 4 passengers. To ensure the feasibility of the instances, the parameters of the EVs—including energy capacities, energy consumption rates, and average speeds—are varied and provided for each instance in the dataset. The release times of rides are not uniformly distributed; in practice, a large number of rides are often requested within short time intervals. Therefore, the distribution of ride request times in this study follows a normal distribution, as applied in a previous study [21]. To simulate a full day's scenario, rides are distributed within the time range [0, 1440], representing the 1440 minutes of a day. The requested time for each ride is generated from the normal distribution $N(720, 360)$, constrained to the range [0, 1440]. Specifically, any generated time value below 0 is truncated to 0, and any value above 1440 is truncated to 1440. Note that this distribution is intended solely for simulation purposes and does not reflect actual empirical release patterns. Ride requests are revealed over time based on their release times, reflecting the problem's online and dynamic nature. At any decision point, only released requests are known, while future requests remain unknown. For each ride request, pickup and drop-off points are randomly paired from the generated customer nodes.

Four instances are generated for Beijing and Chengdu. First, customer node coordinates are sampled based on the heatmaps of customer node distributions in Beijing and Chengdu, respectively. This aims to evaluate how differences in customer distribution and size affect algorithm performance. Subsequently, a baseline scenario (BC) for Beijing (and Chengdu) is generated by randomly creating customer demands, ride release times, and matches between pickup and drop-off points according to the aforementioned probability distributions. Given that real-life customer ride request trends are generally regular and do not fluctuate drastically, we then derived three additional instances based on BC: 1) regenerating customer demand (CD); 2) regenerating ride release time (RT); and 3) rematching pickup and drop-off points (PD). Each of the three instances isolates a specific feature variation from the BC instance, with the modified feature following the same probability distribution as BC. These instances are designed to assess the impact of different stochastic characteristics and to validate the RIP's generality capability, as presented in Section V-D. The instances are named based on their city and

the specific operation applied. For example, BJ-BC refers to the baseline instance for Beijing, while CD-RT represents the instance in Chengdu where the ride release times are adjusted, relative to the CD-BC instance.

B. Experiment Settings

In this study, five algorithms are compared with GEPRIP to validate the effectiveness of the proposed method.

The first two algorithms are based on the max-min ant system (MMAS), which has been shown to outperform other ACO variants in path construction for solving the EVRP [12]. By modifying its objective function and incorporating constraints, MMAS can be adapted to solve the static EVRHM using a direct encoding scheme. MMAS-10 divides the day into 144 time slots (i.e., 10 minutes per slot), enabling it to better accommodate the dynamic characteristics of the EVRHM. When a new ride is released, pickup and drop-off points are initially inserted using a greedy algorithm, and MMAS is called to re-optimize all routes at each time slot interval. In contrast, MMAS-ONTIME re-optimizes the routes every time a new ride is requested. The performance of GEPRIP is compared with MMAS-10 to assess its effectiveness relative to traditional algorithms for solving DEVRLPs. Additionally, the reliability and effectiveness of GEPRIP are further evaluated in comparison to MMAS-ONTIME.

The third comparative algorithm is based on the low-level heuristic template proposed in this study, which utilizes fitness as the heuristic information. This approach, similar to greedy algorithms, is commonly applied to dynamic problems [29] and is referred to as the fitness rule (FR) in this paper. The comparison with FR aims to evaluate how various types of heuristic information affect the performance of solving EVRHM.

The next comparative algorithm is the routing policy (RP) [23], which evolves routing policies and introduces a low-level heuristic template known as the route generation scheme, accompanied by a distinct set of primitives. The comparison aims to analyze the impact of different low-level heuristic templates and primitive sets on the performance of the algorithm.

The final algorithm is GP, which serves as a hyper-heuristic to generate RIPs based on the primitive set used in this study.

TABLE I
PARAMETER SETTINGS FOR GEPRIP

Parameter	Value
Population size	30
Generations	100
Linking function	+
The number of genes	3
The head length of a gene	4
Point mutation rate	0.05
Inversion mutation rate	0.1
IS transpose rate	0.1
RIS transpose rate	0.1
Gene transpose rate	0.1
One-point crossover rate	0.5
Two-point crossover rate	0.2
Gene crossover rate	0.1
Charge threshold	0.3

The parameters for GP are configured according to [25]. By comparing GP with GEPRIP, the goal is to highlight the differences and advantages of GEP in the process of training RIPs.

Based on the commonly used parameter settings from [27] and our preliminary experimental results (detailed in the supplementary material, Section S-B, the parameter settings for the GEPRIP algorithm are provided in Table I. To ensure a fair comparison, the population size and the number of generations for all algorithms, except for FR, are set to 30 and 100, respectively. Other parameters are configured in accordance with their respective References to maintain consistency and credibility.

An extended experiment on the EVRP is also conducted, as detailed in Section S-C of the supplementary material. The positive outcomes in the static environment both provide evidence of the method's accuracy and motivate its application to the more complex dynamic EVRHM environment.

C. Comparison Results

Table II presents the results of 10 independent runs of GEPRIP and other compared algorithms on different test instances, including the minimum, mean, standard deviation, and runtime. Runtimes are reported in seconds (s). The best minimum and mean objective values are highlighted. The symbols “+”, “≈”, and “−” indicate that the performance of GEPRIP is significantly better than, similar to, or worse than other algorithms, based on a Wilcoxon rank-sum test at a significance level of 0.05. From the results, the following observations can be made.

Compared to traditional methods for solving dynamic environments (i.e., MMAS-10 and MMAS-ONTIME), GEPRIP demonstrates superior efficiency and effectiveness in solving EVRHM. In terms of mean values, GEPRIP is only inferior to MMAS-ONTIME in the CD-BC and CD-RT instances according to the statistical test. In other cases, GEPRIP performs comparably to or outperforms both MMAS-10 and MMAS-ONTIME. For the minimum objective values, although MMAS-ONTIME achieves the best-known solutions in five instances, GEPRIP achieves lower minimum values than MMAS-ONTIME in the other three cases, and

outperforms MMAS-10 in six instances. Additionally, the significantly high computational cost of the two MMAS algorithms limits their practicality for real-world EVRHM applications. The results obtained by GEPRIP in a very short time outperform MMAS-10 and are comparable to those of MMAS-ONTIME, even though the two MMAS algorithms require significantly more time. This clearly demonstrates the high performance of GEPRIP. Notably, MMAS-ONTIME does not consistently outperform MMAS-10 in certain test cases, due to MMAS's tendency to get stuck in local optima [12], which also leads to a large standard deviation and poor stability.

Compared to FR and RP, the results highlight the critical role of effective RIPs, and emphasize the efficacy of the insertion-based low-level heuristic template and the primitive set used in GEPRIP. GEPRIP outperforms both FR and RP in terms of both minimum and mean fitness values. Although RP achieves the shortest runtime, the runtime of GEPRIP remains within an acceptable range for dynamic environments. Overall, considering both performance and runtime trade-offs, GEPRIP can be considered superior to both FR and RP.

The comparison between GP and GEPRIP highlights the advantages of GEP in training RIPs. Based on the mean objective values, GEPRIP significantly outperforms GP in four instances. In the other instances, both algorithms exhibit similar competitiveness, as indicated by the Wilcoxon rank-sum test. Regarding the minimum values in Table II, GP outperforms GEPRIP in three test instances, but only significantly in the BJ-CD instance. In the BJ-PD instance, the minimum values of both algorithms are identical, while GEPRIP performs better in the remaining three instances, showing significant improvement in the CD-CD and BJ-BC instances. Furthermore, GP demonstrates poorer stability than GEPRIP in all cases. This is because GP algorithms tend to generate diverse program trees through evolution, which vary greatly in size, shape, and complexity. While this diversity helps explore the solution space, it also leads to significant variations in the performance of the generated RIPs, increasing solution variance [46]. Additionally, since GP typically generates more complex trees, its runtime is slightly longer than GEPRIP. This increased complexity also contributes to overfitting, resulting in poorer generalization of the evolved RIPs by GP compared to GEPRIP, as discussed in more detail in the next subsection.

To further compare these algorithms, Fig. 5 shows the average values of attributes that directly impact solution quality, based on ten runs of each algorithm. These attributes include the number of overdue rides, overtime drivers, dispatched vehicles, and total travel distance. Given the significant difference in the number of overdue rides between the FR algorithm and the others, the y-axis scale in Fig. 5a and Fig. 5e is presented in logarithmic form. From Fig. 5, the following observations can be made.

FR outperforms the other algorithms in terms of total travel distance and shows excellent performance regarding the number of dispatched vehicles. However, it focuses solely on the current optimal solution. This approach neglects a global perspective, overlooking factors such as unpublished rides,

TABLE II
COMPARISON BETWEEN GEPRIP AND OTHER ALGORITHMS ON EIGHT INSTANCES

Network	Instance		MMAS-10	MMAS-ONTIME	FR	RP	GP	GEPRIP
Chengdu	CD-BC	min.	3738.31	3666.35	5435.09	3861.10	3765.20	3780.15
		mean	3843.03($\approx$)	3733.14(-)	5435.09(+)	3914.65(+)	3884.59(+)	3815.09
		std.	77.34	51.91	0.00	26.19	141.74	18.06
		runtime	1180.30	2346.81	2.09	1.92	2.82	2.59
	CD-PD	min.	3674.85	3598.57	5174.92	3785.73	3615.39	3616.15
		mean	3737.53(+)	3720.69($\approx$)	5174.92(+)	3853.51(+)	3754.08(+)	3671.32
		std.	31.81	78.34	0.00	35.27	70.84	27.31
		runtime	1122.40	2298.19	3.21	1.79	2.64	2.50
	CD-RT	min.	3792.54	3670.88	5862.82	3886.16	3739.95	3737.22
		mean	3850.60($\approx$)	3760.59(-)	5862.82(+)	3939.89(+)	3857.40($\approx$)	3825.43
		std.	31.72	58.21	0.00	33.85	66.17	45.80
		runtime	1215.16	2314.63	3.59	1.83	2.97	2.61
	CD-CD	min.	3824.66	3810.72	5583.13	3859.11	3811.47	3604.70
		mean	3931.39(+)	3923.49(+)	5583.13(+)	3923.48(+)	3889.04(+)	3802.15
		std.	99.49	98.84	0.00	84.24	84.73	78.35
		runtime	977.54	2450.56	3.47	1.43	2.99	2.60
Beijing	BJ-BC	min.	11185.66	11004.89	20571.15	11491.62	11205.47	11012.48
		mean	11644.91(+)	11528.29(+)	20571.15(+)	11653.05(+)	11449.37(+)	11237.10
		std.	200.57	319.78	0.00	92.36	305.05	136.06
		runtime	3112.76	6810.25	14.04	7.82	16.00	13.79
	BJ-PD	min.	11143.36	10879.82	20277.38	11593.95	11162.22	11162.22
		mean	11366.04($\approx$)	11332.47($\approx$)	20277.38(+)	11779.51(+)	11386.11($\approx$)	11354.56
		std.	164.60	224.12	0.00	99.04	176.98	157.55
		runtime	2707.81	6175.77	14.26	8.88	13.01	11.84
	BJ-RT	min.	11016.88	10962.09	20458.01	11390.91	10848.26	10742.64
		mean	11550.21(+)	11407.53(+)	20458.01(+)	11544.38(+)	11036.98($\approx$)	10990.67
		std.	255.39	183.84	0.00	100.84	112.36	79.89
		runtime	3183.75	6978.66	9.11	6.49	8.54	8.05
	BJ-CD	min.	11099.42	10849.82	20571.15	11423.85	10680.38	10766.81
		mean	11420.46(+)	11338.88(+)	20571.15(+)	11546.19(+)	11186.61($\approx$)	10977.66
		std.	234.76	252.36	0.00	79.37	308.07	204.58
		runtime	3112.73	7421.48	8.39	6.94	9.06	8.51

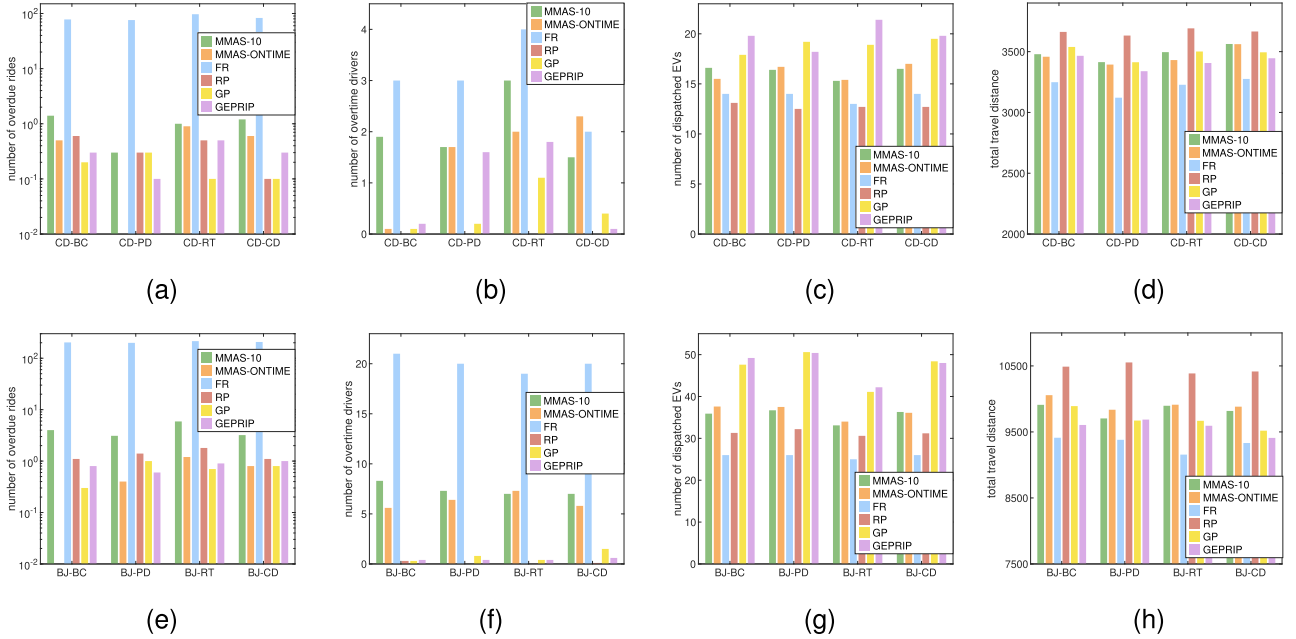


Fig. 5. Features of solutions obtained from GEPRIP and compared algorithms. (a)-(d) represent the attributes of Chengdu instances while (e)-(h) represent those of Beijing.

remaining ride completion times, and the remaining power of vehicles. As a result, it leads to frequent overtime for both drivers and customers. Therefore, it is essential to design effective heuristic information to address these limitations.

Although MMAS-10 is closer to GEPRIP in terms of the number of overdue customers, a slight gap remains, and the number of overtime drivers is significantly higher than GEPRIP. This suggests that more driver overtime is incurred in order to reduce customer overdue times, as the greedy insertion method is initially used when a ride is requested, with optimization occurring only at each time slot interval. In contrast, MMAS-ONTIME performs optimization in real time, but its optimization space is more constrained. During the re-optimization process, the number of inserted rides is limited, and once a vehicle reaches a pickup point, it cannot be replaced. Moreover, both MMAS algorithms lack access to information about future ride requests, which makes the current optimal solution prone to falling into local optima when subsequent rides are considered. Finally, in a dynamic environment, the originally planned routes may need to be adjusted, leading to time wastage. As mentioned earlier, real-time control is a critical requirement in online scheduling systems, where there is limited time to wait for a perfect solution.

The relatively long total travel distance of RP can be attributed to the limitations of its low-level heuristic template and primitive sets. Specifically, the route constructed by RP relies solely on local information, such as the distance to the next node from the current position, without considering global optimization. Furthermore, the template limits RP's ability to proactively dispatch new vehicles; vehicles are only dispatched reactively when customer demands cannot be met. While this approach reduces the number of dispatched vehicles, it fails to compensate for the increased travel distance, leading to lower solution quality compared to GEPRIP. These results highlight the superiority of the insertion-based low-level heuristic template and the globally oriented primitive set used in GEPRIP.

The superior performance of GP and GEPRIP can primarily be attributed to their effectiveness in minimizing the number of overdue customers and drivers. Although this slightly increases the number of dispatched vehicles and the total travel distance, the overall performance remains largely unaffected. This suggests that effective RIPS can balance the various factors that influence solution quality. An analysis of the tree structures of the representative RIPS is presented in Section S-E of the supplementary material.

D. Generality Analysis

In real-world scenarios, daily ride requests vary significantly, making it essential to train RIPS with strong generalization capabilities. This section evaluates the performance of RIPS trained by GP and GEPRIP to assess their generalization ability.

Although the rides vary daily, the ride distribution pattern of the same city generally remains relatively stable. Therefore, the columns GP* and GEPRIP* in Table III represent the performance of ten RIPS trained on CD-BC or BJ-BC across other test instances within the same city. The superior minimum and mean values are highlighted in bold. The symbols “+/ $\approx$ ” indicate that the performance of GEPRIP* is significantly better than or similar to GP* according to a Wilcoxon rank-sum test at a significance level of 0.05.

TABLE III
GENERALITY COMPARISON BETWEEN GEPRIP AND GP

Network	Instance		GP*	GEPRIP*
Chengdu	CD-PD	min.	3746.13	3678.56
		mean	3786.70($\approx$)	3755.15
		std.	72.80	45.55
	CD-RT	min.	3925.39	3862.74
		mean	3981.00(+)	3930.87
		std.	135.78	53.94
Beijing	CD-CD	min.	3835.93	3884.10
		mean	3943.63($\approx$)	3908.42
		std.	122.07	26.93
	BJ-PD	min.	11617.13	11346.77
		mean	12022.21(+)	11665.54
		std.	355.3611	292.6365
	BJ-RT	min.	11250.41	11044.91
		mean	11466.79(+)	11270.59
		std.	362.8747	133.4138
	BJ-CD	min.	11444.32	10985.94
		mean	11566.65(+)	11307.61
		std.	134.4409	309.46

TABLE IV
PARAMETER SETTINGS FOR GENETIC OPERATORS ANALYSIS

	Parameter setting
P1	All operators activate with rates as shown in Table I
P2	Gene crossover rate of 0.7 plus gene transpose rate of 0.2
P3	Only point mutation with a rate of 0.051
P4	Only one-point crossover with a rate of 0.7
P5	Only two-point crossover with a rate of 0.7
P6	Only gene crossover with a rate of 0.7

As shown in Table III, for small-scale instances, the performance of GEPRIP* is comparable to and slightly superior to GP* in two test instances, while significantly better in the remaining instance. For all large-scale cases, GEPRIP* shows marked improvement. In terms of stability, as indicated by the standard deviation, GEPRIP* outperforms GP* in all instances. Overall, GEPRIP demonstrates superior generalization capabilities, particularly for larger and more complex cases.

The primary reason for this is that GP tends to increase in complexity during evolution, leading to expression bloat [47]. GEP mitigates expression bloat through fixed expression length, linear encoding, and an open reading frame [26]. Fig.6 illustrates the lengths of RIPS generated by the GP and GEPRIP algorithms in Chengdu and Beijing. The RIPS generated by GP are significantly longer than those produced by GEPRIP. Complex RIPS tend to overfit, performing well on training instances but poorly on unseen data, a tendency that becomes more pronounced in larger instances. As a result, the relatively simpler RIPS generated by GEPRIP exhibit better generalization ability and are easier to interpret. Furthermore, more complex RIPS require greater computational resources than their simpler counterparts.

E. Genetic Operators Analysis

To investigate the impact of different genetic operations on algorithm performance, Fig.7 compares the average convergence curves of six genetic operations applied to populations of 30 individuals over 100 generations. Each operation was

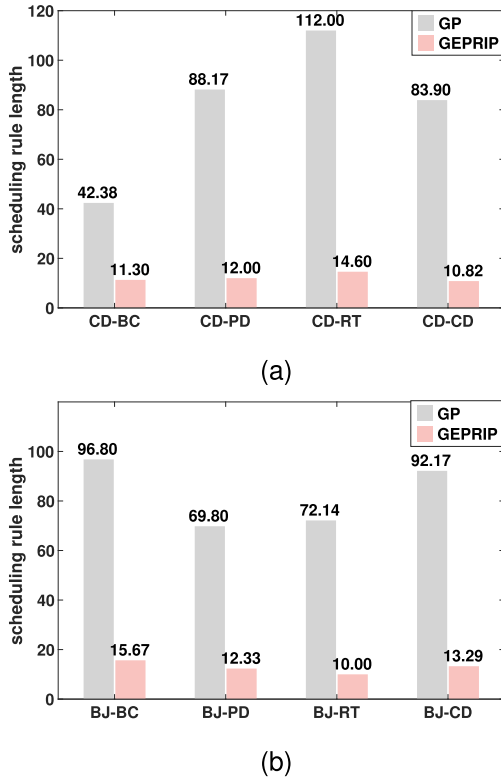


Fig. 6. The length of RIPs trained by GP and GEPRIP. (a) Chengdu. (b) Beijing.

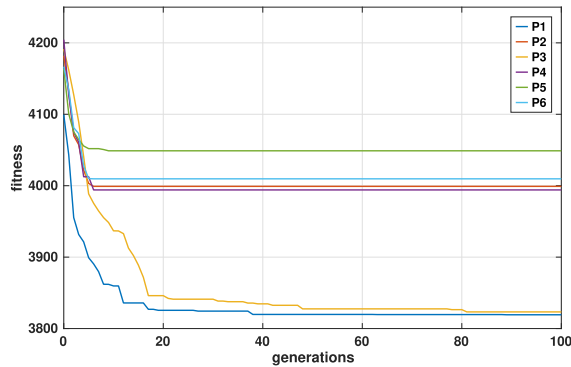


Fig. 7. Convergence curves for six different genetic operators.

run ten times to ensure statistical reliability. The parameters for the genetic operations are sourced from [26], as detailed in Table IV.

Referring to Fig. 7, the performance ranking of the six genetic operation configurations is $P1 > P3 > P4 > P2 > P6 > P5$. Configuration P3, which relies solely on mutation, demonstrates a relatively slow convergence rate. This is due to the absence of crossover, which limits information exchange among individuals and slows the overall search progress. Nevertheless, after sufficient evolutionary iterations, its final performance closely approximates P1. The remaining configurations, which do not incorporate mutation, exhibit comparable performance levels but show a significant performance gap from P1 and P3, displaying signs of premature convergence. These observations highlight the critical role of mutation in

helping the algorithm escape local optima. A closer look at P2 and P6 reveals they only include gene-level operations. In GEP, gene crossover is similar to the recombination in GP, as it only permits the recombination of mathematically concise blocks [26]. Meanwhile, transposition introduces more substantial structural changes in expression trees, potentially facilitating larger jumps in the solution space and uncovering novel patterns. Although P2 incorporates gene transposition in addition to the operations present in P6, the improvement in performance remains limited. This is because gene transposition, confined to the gene-level, is a relatively limited mutation process that only marginally increases population diversity. These findings indicate that relying solely on gene-level operations is insufficient for optimal performance. Symbol-level operations within genes are therefore essential to maintain diversity and avoid premature convergence.

VI. CONCLUSION

This paper aims to accelerate the green transition in the ride-hailing sector by replacing fuel-powered vehicles with EVs. A novel variant of the EVRP, termed the EVRHM, is proposed, providing an excellent fit for the EV fleet ride-pooling services offered by modern ride-hailing companies. This model integrates relevant constraints and reformulates the objective function to better align with the needs of ride-hailing services. To address the dynamic nature of the EVRHM, an online optimization algorithm named GEPRIP is proposed. It establishes an insertion-based low-level heuristic template using a RIP to enable real-time responsiveness. The algorithm's low time complexity reduces the need for ride-hailing companies to invest in significant computational resources and ensures real-time online responses, preventing dispatch delays and long user wait times. By learning the spatiotemporal distribution of rides, a hyper-heuristic based on GEP is proposed to evolve an interpretable RIP that exhibits high performance and strong generalization capabilities, requiring only a single training session. This allows it to automatically adapt to daily fluctuations in order distribution without the need for continuous model retraining. The system is expected to maintain high performance even during sudden spikes in demand, such as during heavy rain or after a concert. Experimental results demonstrate that GEPRIP significantly outperforms traditional methods in handling dynamic ride requests, providing more generalizable solutions than GP.

While the current work provides valuable insights, several limitations remain, which we plan to address in future research. From the perspective of the problem model, real-time prices and power capacities at different charging stations vary in real-world scenarios. We aim to integrate the power grid with the road network in future work.

With the ongoing electrification and intelligent transformation of transportation systems, the applicability of this research is extended to a range of real-world scenarios rather than being limited to ride-hailing services. The proposed model and algorithm can be adapted to other real-world scenarios, such as electric logistics fleets and uncrewed aerial vehicle delivery. To further explore its application in these real-world scenarios, a promising extension of the method involves

a distributed and decentralized framework where decision-making is delegated to sub-regions [48], [49]. Moreover, a multi-agent collaborative framework can be utilized [50], [51], evolving differentiated rules as reinforcement learning actions for intelligent agents with various roles in the system [52].

REFERENCES

- [1] R. Sims et al., *Transport Climate Change 2014: Mitigation of Climate Change. Contribution Of Working Group III to the Fifth Assessment Report of the Intergovernmental Panel on Climate Change*, O. Edenhofer et al., Ed., Cambridge, U.K.: Cambridge Univ. Press, 2014. [Online]. Available: http://www.ipcc.ch/pdf/assessment-report/ar5/wg3/ipcc_wg3_ar5_chapter8.pdf
- [2] S. Zhou, J. Chen, Z. Wu, and Y. Qiu, "Electrification of online ride-hailing vehicles in China: Intention modelling and market prediction," *Energies*, vol. 14, no. 21, p. 7380, Nov. 2021.
- [3] W.-L. Liu, Y.-J. Gong, W.-N. Chen, Z. Liu, H. Wang, and J. Zhang, "Coordinated charging scheduling of electric vehicles: A mixed-variable differential evolution approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 12, pp. 5094–5109, Dec. 2020.
- [4] Y.-H. Jia, Y. Mei, and M. Zhang, "A bilevel ant colony optimization algorithm for capacitated electric vehicle routing problem," *IEEE Trans. Cybern.*, vol. 52, no. 10, pp. 10855–10868, Oct. 2022.
- [5] M. Mavrouniotis et al., "Benchmark set for the IEEE WCCI-2020 competition on evolutionary computation for the electric vehicle routing problem," in *Proc. KIOS COE*, 2020, pp. 1–7.
- [6] R. Basso, B. Kulcsár, I. Sanchez-Díaz, and X. Qu, "Dynamic stochastic electric vehicle routing with safe reinforcement learning," *Transp. Res. E, Logistics Transp. Rev.*, vol. 157, Jan. 2022, Art. no. 102496.
- [7] T. Chen, B. Zhang, H. Pourbabak, A. Kavousi-Fard, and W. Su, "Optimal routing and charging of an electric vehicle fleet for high-efficiency dynamic transit systems," *IEEE Trans. Smart Grid*, vol. 9, no. 4, pp. 3563–3572, Jul. 2018.
- [8] M. Schneider, A. Stenger, and D. Goetze, "The electric vehicle-routing problem with time windows and recharging stations," *Transp. Sci.*, vol. 48, no. 4, pp. 500–520, Nov. 2014.
- [9] Z. Liu, X. Zuo, M. Zhou, W. Guan, and Y. Al-Turki, "Electric vehicle routing problem with variable vehicle speed and soft time windows for perishable product delivery," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 6, pp. 6178–6190, Jun. 2023.
- [10] C. Yao, S. Chen, and Z. Yang, "Online distributed routing problem of electric vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 9, pp. 16330–16341, Sep. 2022.
- [11] J. Zhao, M. Mao, X. Zhao, and J. Zou, "A hybrid of deep reinforcement learning and local search for the vehicle routing problems," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 11, pp. 7208–7218, Nov. 2021.
- [12] Y.-H. Jia, Y. Mei, and M. Zhang, "Confidence-based ant colony optimization for capacitated electric vehicle routing problem with comparison of different encoding schemes," *IEEE Trans. Evol. Comput.*, vol. 26, no. 6, pp. 1394–1408, Dec. 2022.
- [13] J. P. Vielma, "Mixed integer linear programming formulation techniques," *SIAM Rev.*, vol. 57, no. 1, pp. 3–57, Jan. 2015.
- [14] H. Tahami, G. Rabadi, and M. Haouari, "Exact approaches for routing capacitated electric vehicles," *Transp. Res. E, Logistics Transp. Rev.*, vol. 144, Dec. 2020, Art. no. 102126.
- [15] M. Keskin and B. Çatay, "A matheuristic method for the electric vehicle routing problem with time windows and fast chargers," *Comput. Oper. Res.*, vol. 100, pp. 172–188, Dec. 2018.
- [16] J. Hof, M. Schneider, and D. Goetze, "Solving the battery swap station location-routing problem with capacitated electric vehicles using an AVNS algorithm for vehicle-routing problems with intermediate stops," *Transp. Res. B, Methodol.*, vol. 97, pp. 102–112, Mar. 2017.
- [17] Y. Wang, J. Bi, W. Guan, and X. Zhao, "Optimising route choices for the travelling and charging of battery electric vehicles by considering multiple objectives," *Transp. Res. D, Transp. Environ.*, vol. 64, pp. 246–261, Oct. 2018.
- [18] R. Montemanni, L. M. Gambardella, A. E. Rizzoli, and A. V. Donati, "Ant colony system for a dynamic vehicle routing problem," *J. Combinat. Optim.*, vol. 10, no. 4, pp. 327–343, Dec. 2005.
- [19] P. Kilby, P. Prosser, and P. Shaw, "Dynamic VRPs: A study of scenarios," Univ. Strathclyde, Glasgow, U.K., Tech. Rep. APES-06-1998, 1998.
- [20] X. Xiang, Y. Tian, X. Zhang, J. Xiao, and Y. Jin, "A pairwise proximity learning-based ant colony algorithm for dynamic vehicle routing problems," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 6, pp. 5275–5286, Jun. 2022.
- [21] Y.-H. Jia et al., "A dynamic logistic dispatching system with set-based particle swarm optimization," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 48, no. 9, pp. 1607–1621, Sep. 2018.
- [22] W. Jian-Xin, "Research on vehicle routing problem for the dynamic needs of IoT customers," Ph.D. dissertation, Chongqing Univ., Chongqing, China, 2021, doi: [10.27670/d.cnki.gcqdu.2021.000646](https://doi.org/10.27670/d.cnki.gcqdu.2021.000646).
- [23] F. J. Gil-Gala, M. Durasević, and D. Jakobović, "Evolving routing policies for electric vehicles by means of genetic programming," *Appl. Intell.*, vol. 54, no. 23, pp. 12391–12419, Dec. 2024, doi: [10.1007/s10489-024-05803-5](https://doi.org/10.1007/s10489-024-05803-5).
- [24] F. J. G. Gala, M. Durasević, and D. Jakobović, "Genetic programming for electric vehicle routing problem with soft time windows," in *Proc. Genetic Evol. Comput. Conf. Companion*, Jul. 2022, pp. 542–545, doi: [10.1145/3520304.3528994](https://doi.org/10.1145/3520304.3528994).
- [25] X.-C. Liao, Y. Mei, and M. Zhang, "Learning traffic signal control via genetic programming," in *Proc. Genetic Evol. Comput. Conf.*, Jul. 2024, pp. 924–932.
- [26] C. Ferreira, "Gene expression programming: A new adaptive algorithm for solving problems," 2001, *arXiv: cs/0102027*.
- [27] X.-C. Liao, W.-N. Chen, Y.-H. Jia, and W.-J. Qiu, "Towards scalable dynamic traffic assignment with streaming agents: A decentralized control approach using genetic programming," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 8, no. 1, pp. 942–955, Feb. 2024.
- [28] X.-C. Liao, X.-M. Hu, X.-L. Chen, Y. Mei, Y.-H. Jia, and W.-N. Chen, "Learning to communicate among agents for large-scale dynamic path planning with genetic programming hyperheuristic," *IEEE Trans. Artif. Intell.*, vol. 6, no. 5, pp. 1269–1283, May 2025.
- [29] X.-C. Liao, Y.-H. Jia, X.-M. Hu, and W.-N. Chen, "Uncertain commuters assignment through genetic programming hyper-heuristic," *IEEE Trans. Computat. Social Syst.*, vol. 11, no. 2, pp. 2606–2619, Apr. 2024.
- [30] C. Ye, W. He, and H. Chen, "Electric vehicle routing models and solution algorithms in logistics distribution: A systematic review," *Environ. Sci. Pollut. Res.*, vol. 29, no. 38, pp. 57067–57090, Aug. 2022.
- [31] R. G. Conrad and M. Figliozzi, "The recharging vehicle routing problem," in *Proc. 61st Annu. IIE Conf.*, Jan. 2011, pp. 1–8.
- [32] S. Erdogan and E. Miller-Hooks, "A green vehicle routing problem," *Transp. Res. E, Logistics Transp. Rev.*, vol. 48, no. 1, pp. 100–114, Jan. 2011.
- [33] M. W. P. Savelsbergh and M. Sol, "The general pickup and delivery problem," *Transp. Sci.*, vol. 29, no. 1, pp. 17–29, Feb. 1995.
- [34] Y. Yilmaz and C. B. Kalayci, "Variable neighborhood search algorithms to solve the electric vehicle routing problem with simultaneous pickup and delivery," *Mathematics*, vol. 10, no. 17, p. 3108, Aug. 2022.
- [35] S. Wang, Y. Liu, X. Huang, and W. Sun, "Adaptive large neighborhood search for the dynamic vehicle routing problem with electric vehicles," in *Proc. 35th Chin. Control Decis. Conf. (CCDC)*, May 2023, pp. 3776–3780.
- [36] S. Zhang, Y. Gajpal, S. S. Appadoo, and M. M. S. Abdulkader, "Electric vehicle routing problem with recharging stations for minimizing energy consumption," *Int. J. Prod. Econ.*, vol. 203, pp. 404–413, Sep. 2018.
- [37] F. W. Glover and G. A. Kochenberger, *Handbook of Metaheuristics*, vol. 57. Boston, MA, USA: Springer, 2006.
- [38] G. Zhenfeng, L. Yang, J. Xiaodan, and G. Sheng, "The electric vehicle routing problem with time windows using genetic algorithm," in *Proc. IEEE 2nd Adv. Inf. Technol., Electron. Autom. Control Conf. (IAEAC)*, Mar. 2017, pp. 635–639.
- [39] A. Montoya, C. Guéret, J. E. Mendoza, and J. G. Villegas, "The electric vehicle routing problem with nonlinear charging function," *Transp. Res. B, Methodol.*, vol. 103, pp. 87–110, Sep. 2017.
- [40] E. K. Burke, M. Hyde, G. Kendall, G. Ochoa, E. Özcan, and J. R. Woodward, "A classification of hyper-heuristic approaches," in *Handbook of Metaheuristics*, M. Gendreau and J.-Y. Potvin, Eds., Boston, MA, USA: Springer, 2010, pp. 449–468.
- [41] J. Koza, "Genetic programming as a means for programming computers by natural selection," *Statist. Comput.*, vol. 4, no. 2, pp. 87–112, 1994.
- [42] B. Xu, Y. Mei, Y. Wang, Z. Ji, and M. Zhang, "Genetic programming with delayed routing for multiobjective dynamic flexible job shop scheduling," *Evol. Comput.*, vol. 29, no. 1, pp. 75–105, Mar. 2021.
- [43] S. Zhang and X. Liu, "Prediction of relations between leaching rate of rare Earth mineral and the density of its mother liquid based on HGEP-PTS algorithm," *Microelectron. Rel.*, vol. 93, pp. 115–119, Feb. 2019.
- [44] A. Froger, J. E. Mendoza, O. Jabali, and G. Laporte, "Improved formulations and algorithmic components for the electric vehicle routing problem with nonlinear charging functions," *Comput. Oper. Res.*, vol. 104, pp. 256–294, Apr. 2019.

- [45] Y. Yang, G. Chen, H. Ma, S. Hartmann, and M. Zhang, "Dual-tree genetic programming with adaptive mutation for dynamic workflow scheduling in cloud computing," *IEEE Trans. Evol. Comput.*, early access, Apr. 24, 2025, doi: [10.1109/TEVC.2024.3392968](https://doi.org/10.1109/TEVC.2024.3392968).
- [46] B. Burlacu, K. Yang, and M. Affenzeller, "Population diversity and inheritance in genetic programming for symbolic regression," *Natural Comput.*, vol. 23, no. 3, pp. 531–566, Sep. 2024.
- [47] S. Silva and E. Costa, "Dynamic limits for bloat control in genetic programming and a review of past and current bloat theories," *Genetic Program. Evolvable Mach.*, vol. 10, no. 2, pp. 141–179, Jun. 2009.
- [48] F.-F. Wei, W.-N. Chen, X.-Q. Guo, B. Zhao, S.-W. Jeon, and J. Zhang, "CrowdEC: Crowdsourcing-based evolutionary computation for distributed optimization," *IEEE Trans. Services Comput.*, vol. 17, no. 6, pp. 3286–3299, Nov. 2024.
- [49] W. Chen, F.-F. Wei, T.-F. Zhao, K. C. Tan, and J. Zhang, "A survey on distributed evolutionary computation," *IEEE Comput. Intell. Mag.*, pp. 41–62, Aug. 2023.
- [50] T.-Y. Chen et al., "The confluence of evolutionary computation and multi-agent systems: A survey," *IEEE-CAA J. Autom. Sin.*, vol. 12, pp. 1–19, 2025.
- [51] T.-Y. Chen, W.-N. Chen, J.-K. Hao, Y. Wang, and J. Zhang, "Multi-agent evolution strategy with cooperative and cumulative step adaptation for black-box distributed optimization," *IEEE Trans. Evol. Comput.*, early access, Jan. 6, 2025, doi: [10.1109/TEVC.2025.3525713](https://doi.org/10.1109/TEVC.2025.3525713).
- [52] X. Chen, R. Bai, R. Qu, J. Dong, and Y. Jin, "Deep reinforcement learning assisted genetic programming ensemble hyper-heuristics for dynamic scheduling of container port trucks," *IEEE Trans. Evol. Comput.*, vol. 29, no. 4, pp. 1371–1385, Aug. 2025.



Ming-Chu Yang received the bachelor's degree from the School of Computer and Communication Engineering, Northeastern University at Qinhuangdao, China, in 2024. He is currently pursuing the master's degree with the School of Computer Science and Engineering, South China University of Technology, Guangzhou, China.

His current research interests include evolutionary computation, swarm intelligence, reinforcement learning, and their applications in intelligent transportation.



Wei-Neng Chen (Senior Member, IEEE) received the bachelor's and Ph.D. degrees in computer science from Sun Yat-sen University, Guangzhou, China, in 2006 and 2012, respectively.

He is currently a Full Professor with the School of Computer Science and Engineering, South China University of Technology, Guangzhou. He has co-authored over 200 international journal and conference papers, including more than 100 articles published in IEEE TRANSACTIONS journals. His current research interests include computational intelligence, swarm intelligence, network science, and their applications.

Prof. Chen is a Committee Member of the IEEE CIS Emerging Topics Task Force. He is the Vice Chair of the IEEE Guangzhou Section and the Chair of the IEEE SMC Society Guangzhou Chapter. He serves as an Associate Editor for IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION.



Feng-Feng Wei (Member, IEEE) received the bachelor's and Ph.D. degrees in computer science and technology from South China University of Technology, Guangzhou, China, in 2019 and 2023, respectively.

She is currently an Assistant Professor with the School of Computer Science and Engineering, South China University of Technology. Her current research interests include swarm intelligence, evolutionary computation, distributed optimization, edge-cloud computing, and their applications on

data-driven optimization in real-world problems.



Jun Zhang (Fellow, IEEE) received the Ph.D. degree in electrical engineering from the City University of Hong Kong in 2002.

His research activities are mainly in the areas of computational intelligence (CI). Based on his research in evolutionary computation (EC) and its applications, he has published more than 600 peer-reviewed research articles, of which more than 230 have been published in IEEE TRANSACTIONS. He is a Clarivate Highly Cited Researcher, ranked in the top 1% for the field of computer science, and

was awarded the Outstanding Young Scientist Fund by NSFC in 2011. He serves as an Associate Editor for IEEE TRANSACTIONS ON ARTIFICIAL INTELLIGENCE and IEEE TRANSACTIONS ON CYBERNETICS.